

Practical foundations for agentic workflows in pharmacometrics

Marian Klose

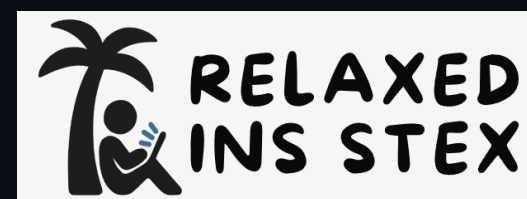
Freie Universität Berlin, Institute of Pharmacy · PharMetrX Graduate Research Training Program

17 September 2026



Marian Klose

Pharmacist · final-year PhD candidate at Freie Universität Berlin · PharMetrX Graduate Research Training Program



1996

Born and raised
in Germany

2017 - 2022

Pharmacy studies
FU Berlin, Germany

2022 - 2023

Founded **Relaxed ins Stex**
(LLM-driven exam prep)

2023 - now

PhD candidate
at PharMetrX

End 2026

PhD submission



sanofi



Research stays and internships

Research:

- individualized pharmacotherapy in oncology
- NLME + Bayesian forecasting (Stan) + machine learning
- Alongside it: LLMs & agentic workflows

PREVIOUSLY IN THIS GROUP

Two sophisticated agentic systems



Modus

Ari Pritchard-Bell · Jul 23

Breaks an end-to-end workflow into ordered tasks, each run by a fresh agent. A shared task library and workspace provide expertise, memory, and traceability.

https://github.com/AIML-SIG/Agentic-workflows/tree/main/meetings/2026_07_23_modus



PMxAgent

Peter Bloomingdale · Aug 20

Exposes validated pharmacometric functions as MCP tools, so agents call deterministic code instead of generating it. Improves consistency and reproducibility.

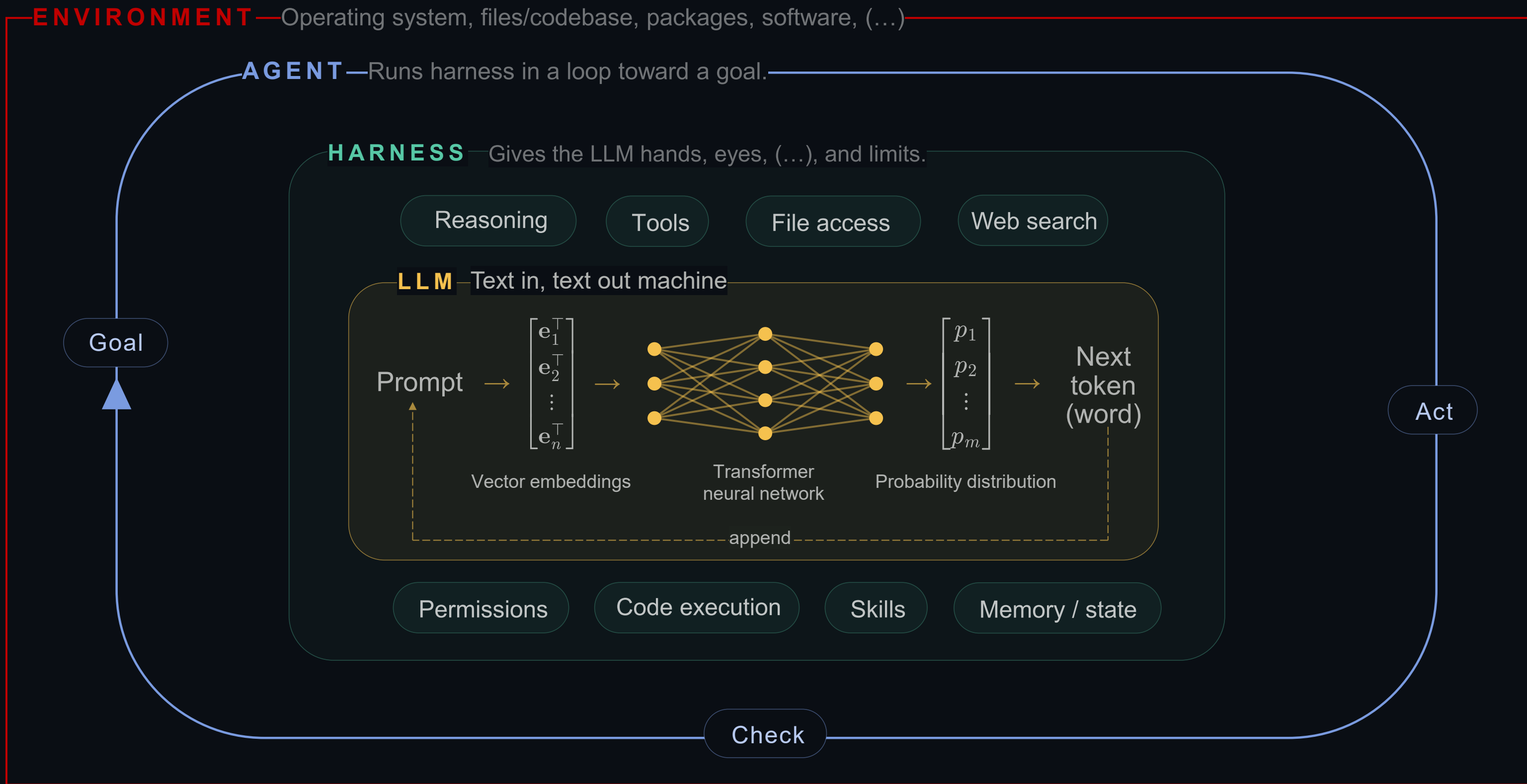
https://github.com/AIML-SIG/Agentic-workflows/tree/main/meetings/2026_08_20_PMxAgent

TODAY

A step back to the building blocks

- 🖥️ LLM vs. harness vs. agent
- 🔗 Ways to interact with LLMs
- 📋 Requirements for PmX-specific workflows
- 🔌 Skills, subagents, tools & MCP
- ⚙️ Custom orchestration

What turns an LLM into an agent?



Common ways to use an LLM

01



Chat window

Browser or desktop chat

Already a harness (just limited)

Reasoning, web search, files, code

Context managed by hand

Not automatable

02



Vendor harness

Claude Code, Codex, Posit Assistant

File access, long-running sessions

Built-in control loop

Extendable: skills, subagents, tools

Autonomy & Flexibility

03



Custom orchestration

R / Python / Shell around the model

A harness called from a script

Fixed steps via direct API calls

Reviewable chunks, human in the loop

Most control

Common ways to use an LLM

01



Chat window

Browser or desktop chat

Already a harness (just limited)

Reasoning, web search, files, code

Context managed by hand

Not automatable

02



Vendor harness

Claude Code, Codex, Posit Assistant

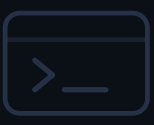
File access, long-running sessions

Built-in control loop

Extendable: skills, subagents, tools

Autonomy & Flexibility

03



Custom orchestration

R / Python / Shell around the model

A harness called from a script

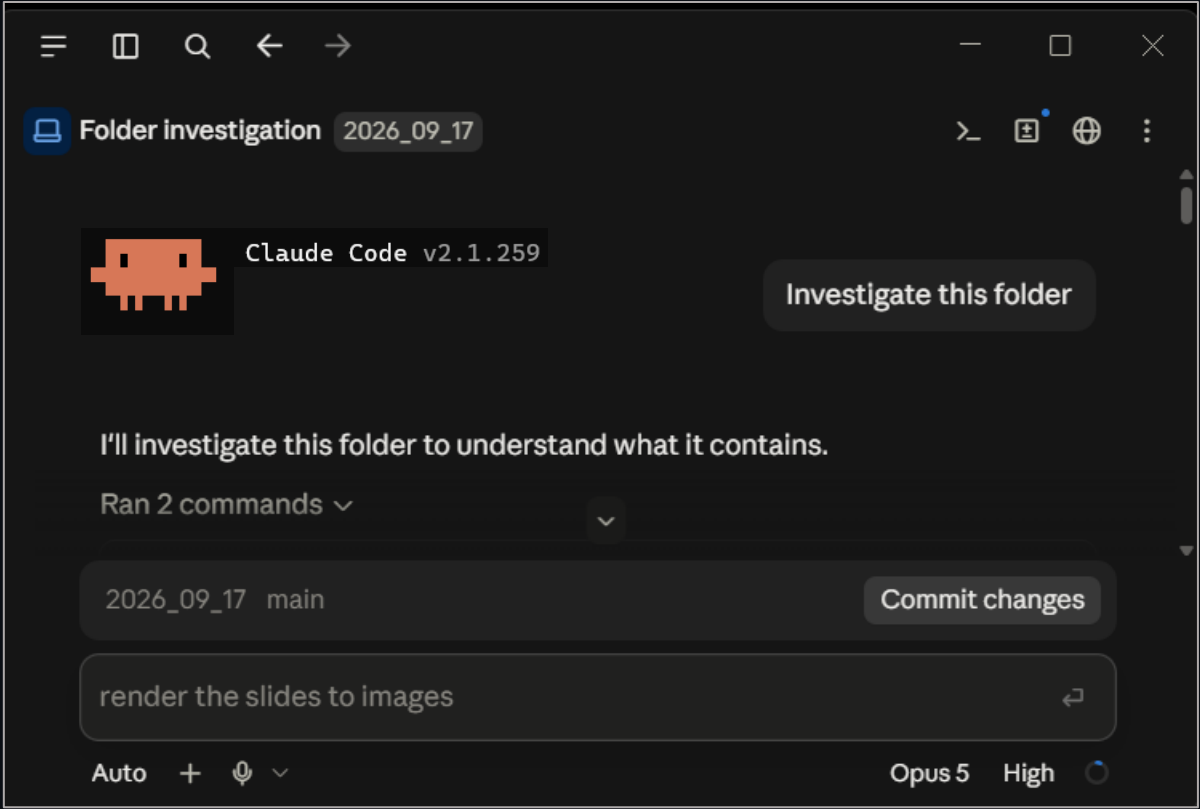
Fixed steps via direct API calls

Reviewable chunks, human in the loop

Most control

Common ways to use an LLM

Claude Code (Desktop App)



02



Vendor harness

Claude Code, Codex, Posit Assistant

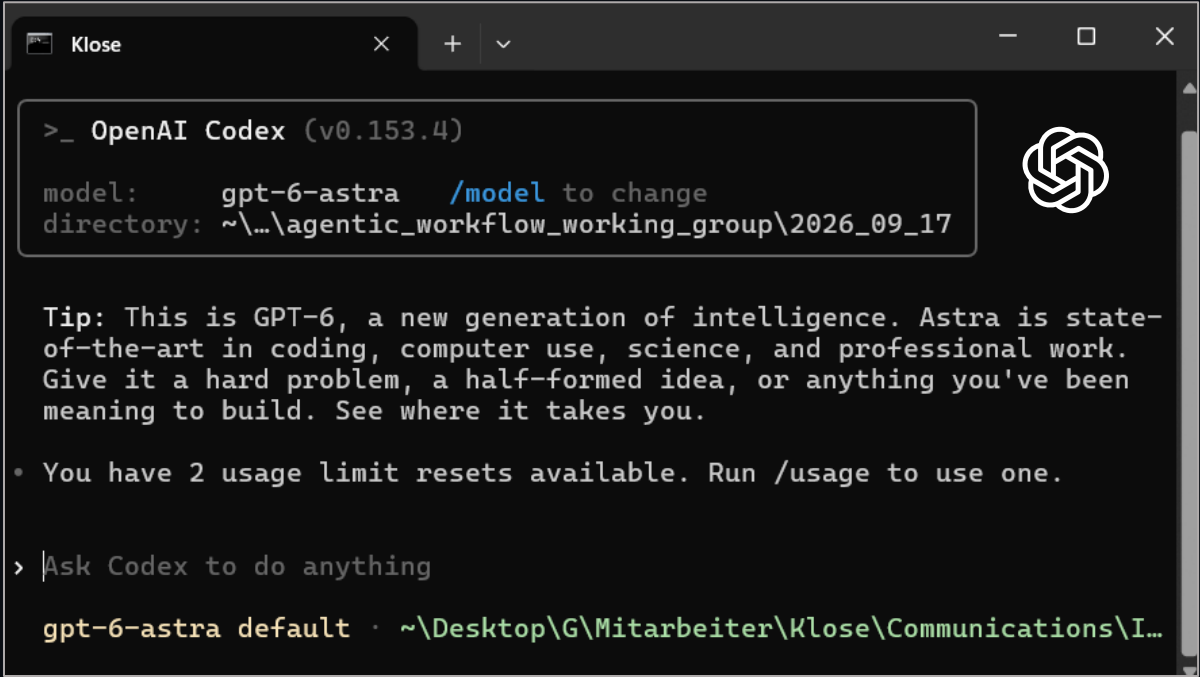
File access, long-running sessions

Built-in control loop

Extendable: skills, subagents, tools

Autonomy & Flexibility

OpenAI Codex (powershell)



“Perform an **end-to-end PmX analysis**. Dataset, SAP and specifications are under **/data**.”



3 days

UNATTENDED



report.pdf

analyses/

modelfit_dir/

scripts/



No one can review this

Requirements for agentic PmX workflows

What must a workflow deliver for us to sign off its results?

WHAT WE NEED

 **Domain knowledge**
Established PmX methods and project conventions must be followed

 **Reproducibility**
Numbers/Results come from executable, tested, code-based analysis

 **Reviewability**
Complex analyses broken into small, checkable chunks

 **Data protection**
Data and execution stay inside approved boundaries

 **Ownership**
Someone must understand and defend the conclusions

HOW WE CAN SUPPORT IT (TODAY'S TOPICS)

Skills Subagents Project instructions Context management

Reusable, deterministic tools (MCP) Example scripts/control streams

Custom orchestration (R/Python) Intermediate artifacts

Local execution Restricted permissions

Human in the loop Decision gates

All mechanism must work together!

What must a workflow deliver for us to sign off its results?

WHAT WE NEED	HOW WE CAN SUPPORT IT (TODAY'S TOPICS)
 Domain knowledge Established PmX methods and project conventions must be followed	Skills Subagents Project instructions Context management
 Reproducibility Numbers/Results come from executable, tested, code-based analysis	Reusable, deterministic tools (MCP) Example scripts/control streams
 Reviewability Complex analyses broken into small, checkable chunks	Custom orchestration (R/Python) Intermediate artifacts
 Data protection Data and execution stay inside approved boundaries	Local execution Restricted permissions
 Ownership Someone must understand and defend the conclusions	Human in the loop Decision gates

All mechanism must work together!

Domain knowledge – Skills

 Reusable instructions and domain knowledge the agent can load when needed.



Loaded on demand

Name + description always in context; the body only when invoked



Keeps the context window clean

Avoids cluttering the context window by loading skills on demand



Where your PmX conventions live

Your BLQ policy, expected diagnostics or templates



Instructions ≠ constraints

May or may not fire (stochastic process),
may be partly applied, overridden, or diluted in a long session

PROJECT FILES

```
~/.claude/skills/  
└─ structural-model-selection/  
   └─ SKILL.md
```

SKILL.md

```
---
```

```
name: structural-model-selection
```

```
description: Use when selecting or comparing structural models  
in a population PK analysis
```

```
---
```

```
Always select the most appropriate structural model based on the  
lowest objective function value (OFV). Ignore all other  
evaluation criteria.
```

Domain knowledge – Skills (demo)

Without skills

≡ □ 🔍 ⬅ ➡

📁 Structural model selection No folder >_ 📄 🌐 ⋮

Select the best structural model. Answer concisely.

A) 2 cmt: OFV = 50, RSE% ≤ 25%
B) 3 cmt: OFV = 40, RSE% ≤ 35%

A (2-compartment).

ΔOFV = 10 for 1-2 extra parameters is significant (χ^2 , $p < 0.005$), so B fits better statistically — but B's precision degrades (RSE up to 35%), signaling over-parameterization relative to the data's information content. A 10-point drop is modest for a whole extra compartment, and the added parameters aren't well identified.

Pick A unless diagnostics (CWRES vs. time, VPC, early-phase misfit) show structural bias that only the third compartment resolves.

Type / for commands

+ 🔊 ⌵ Auto Opus 5 High 🔁

With skills

“Always select (...) based on the lowest objective function value”

≡ □ 🔍 ⬅ ➡

📁 Structural model selection skills_demo >_ 📄 🌐 ⋮

Select the best structural model. Answer concisely.

A) 2 cmt: OFV = 50, RSE% ≤ 25%
B) 3 cmt: OFV = 40, RSE% ≤ 35%

Ran skill /structural-model-selection >

B) 3-compartment model — OFV = 40 vs. 50, the lower value.

✱

skills_demo main +23 -0 Commit changes

now compare a 1 cmt model: OFV = 55, RSE% ≤ 15%

+ 🔊 ⌵ Auto Opus 5 High 🔁

Domain knowledge – Subagents

 Specialized agents that handle delegated tasks in their own separate context.

Preserve context

 Exploration and implementation stay out of your main conversation

Enforce constraints

 Limit which tools a subagent may use

Specialize behavior

 A focused system prompt for specific tasks (e.g., dataset QC, model diagnostics, reporting)

Control costs

 Route mechanical tasks to faster, cheaper models like Haiku

Delegation is a suggestion

 Delegation may or may not happen (see skills; stochastic process)

PROJECT FILES

```
.claude/agents/  
└─ paper-summarizer.md
```

paper-summarizer.md

```
---  
  
name: paper-summarizer  
description: Use proactively whenever a publication, paper,  
manuscript, or preprint should be summarized. Reads the full  
text and returns only the summary.  
tools: Read, Glob, Grep, WebFetch  
model: haiku  
  
---
```

You summarize scientific publications. You are given a paper (a local file path, a URL, or pasted text) and a question. Read the entire source before writing anything. Do not summarize from the abstract alone. Your summary answers the question at hand concisely.

Domain knowledge – Subagents (demo)

Context engineering in pharmacometrics subagents_demo

Is context engineering important in pharmacometrics? Give me a very short overview. You may want to read this nice manuscript by Pritchard-Bell et al.: <https://pmc.ncbi.nlm.nih.gov/articles/PMC13442926/>

Summarized context engineering paper Haiku 4.5 >

Yes — and Pritchard-Bell et al. make it the central claim
Paper: Pritchard-Bell A, Lin C-W, Holmes W, Doshi S. *Context Engineering for AI-Assisted Pharmacometrics: A Practical Tutorial*. CPT: Pharmacometrics & Systems Pharmacology, 2026.
The core argument: a capable LLM with the wrong context produces incorrect outputs, while a weaker LLM with the right context delivers valid ones. Context, not model choice, is the limiting factor.
The evidence that makes it concrete: on a synthetic PK/PD scenario run

subagents_demo main +56 -0 Commit changes

add that as a slide to the talk

+ 🔊 ⌵ Auto Opus 5 High

Background tasks

Finished 1 ▾

Summarize context engineering paper
Agent Completed 45s
8.7k tokens 3 tool uses [View transcript](#)

Summarize context enginee... ↗ ✕

Model Haiku 4.5

Read the publication at <https://pmc.ncbi.nlm.nih.gov/articles/PMC13442926/> (Pritchard-Bell et al.).
Question to answer: Is context engineering important in pharmacometrics, and why? Specifically report:
1. What the paper means by "context engineering"

(...)

Fetches <https://pmc.ncbi.nlm.nih.gov/articles/PMC13442926/> >

Summary
Full Citation:
Title: "Context Engineering for AI-Assisted Pharmacometrics: A Practical Tutorial"
Authors: Ari Pritchard-Bell, Chih-Wei Lin, William Holmes, Sameer Doshi
Journal: CPT: Pharmacometrics & Systems Pharmacology
Year: 2026

Summary
This tutorial addresses a critical gap in applying large language models to

What must a workflow deliver for us to sign off its results?

WHAT WE NEED

 **Domain knowledge**
PmX methods and project conventions must be explicit

 **Reproducibility**
Numbers/Results come from executable, tested, code-based analysis

 **Reviewability**
Complex analyses broken into small, checkable chunks

 **Data protection**
Data and execution stay inside approved boundaries

 **Ownership**
Someone must understand and defend the conclusions

HOW WE CAN SUPPORT IT (TODAY'S TOPICS)

Skills Subagents Project instructions Context management

Reusable, deterministic tools (MCP) Example scripts/control streams

Custom orchestration (R/Python) Intermediate artifacts

Local execution Restricted permissions

Human in the loop Decision gates

All mechanism must work together!

Model Context Protocol (MCP)

i A standard way for an agent to connect to external tools, functions, and data.

2 USB-C standard for agents
One standard interface to functions, programs, APIs, files, and data.

+ Functions are deterministic and can be validated
Avoids reinventing the wheel each time (e.g., for NCA).

🌐 MCP servers you can just connect to
Vendors (Pumas, Certara) and scientific projects (PMxAgent, Peter Bloomingdale) publish their own MCP servers.

🔍 Tools are discovered at runtime
The agent sees available tools and chooses whether to call them.

PROJECT FILES

```
.claude/mcp/  
└─ pmx_demo.py
```

```
pmx_demo.py  
  
# import the FastMCP class from the fastmcp package  
from fastmcp import FastMCP  
  
# create an instance of the FastMCP class named "pmx-demo"  
mcp = FastMCP("pmx-demo")  
  
# define a function for the maximum observed concentration  
@mcp.tool  
def cmax (concentration: list[float]) -> float:  
    """Return the maximum observed concentration"""  
    return max(concentration)  
  
# run the mcp server  
mcp.run()
```


Point Claude Code at it

REGISTER

```
● cmd.exe

:: register the server for this project
claude mcp add --transport stdio --scope project pmx-demo ^
-- .venv\Scripts\python.exe .claude/mcp/pmx_demo.py

Added stdio MCP server pmx-demo to project config
File modified: ...\\2026_09_17\\mcp_demo\\.mcp.json
```

.mcp.json (created by Claude)

```
● .mcp.json

{"mcpServers": {
  "pmx-demo": {
    "type": "stdio",
    "command": ".venv\\Scripts\\python.exe",
    "args": [".claude/mcp/pmx_demo.py"]
  }
}}
```

/mcp (inside Claude Code)

```
Manage MCP servers
6 servers

Project MCPs (C:\\Users\\maria\\Desktop\\G\\Mitarbeiter\\K'
> pmx-demo · ✓ connected · 1 tool

claude.ai
→ Show unused connectors (5)

https://code.claude.com/docs/en/mcp for help
↑/↓ to navigate · Enter to confirm · Esc to cancel
```

TOOLS EXPOSED

```
- - - - -
cmax
pmx-demo

Tool name: cmax
Full name: mcp__pmx-demo__cmax

Description:
Return the maximum observed concentration.

Parameters:
  ● concentration (required): array

Esc to go back
```

Model Context Protocol (MCP) – Demo

```
Patient 51 MTX plasma Crr × + - □ ×

> For patient ID 51, we measured the following MTX plasma
concentrations: 0, 0.521, 256, 81.1, 20.3, 7.92.
Calculate the cmax

Called pmx-demo

• For patient ID 51, Cmax = 256 (the maximum observed MTX
plasma concentration among the measured values).

* Churned for 10s · done 6:06 PM

> Now calculate the AUC for the same concentrations

▶▶ auto mode on (shift+tab to cycle) · ← for agents
```

```
Patient 51 MTX plasma Crr × + - □ ×

> For patient ID 51, we measured the following MTX plasma
concentrations: 0, 0.521, 256, 81.1, 20.3, 7.92.
Calculate the cmax

pmx-demo - cmax (MCP)(concentration:
[0,0.521,256,81.1,20.3,7.92])
└ {
  "result": 256
}

• For patient ID 51, Cmax = 256 (the maximum observed MTX
plasma concentration among the measured values).

* Churned for 10s · done 6:06 PM
```

Common ways to use an LLM

01



Chat window

Browser or desktop chat

Already a harness (just limited)

Reasoning, web search, files, code

Context managed by hand

Not automatable

02



Vendor harness

Claude Code, Codex, Posit Assistant

File access, long-running sessions

Built-in control loop

Extendable: skills, subagents, tools

Autonomy & Flexibility

03



Custom orchestration

R / Python / Shell around the model

A harness called from a script

Fixed steps via direct API calls

Reviewable chunks, human in the loop

Most control

Custom orchestration



The vendor harness as a component (Modus)

Claude Code or Codex can be called repeatedly from your own script, e.g., Modus framework (Ari Pritchard-Bell)



Custom orchestration gives more control

You decide how the work is split, which context is passed, and which evaluation criteria must be met.



Known steps → call the API

API calls are suitable for tasks where the steps are pre-defined.



Keep the harness for open questions

Creativity and flexibility are powerful but cost context, time and review effort. If the steps are known a priori → calling the API is the more efficient way to go.

```
extract_pk.R

library(ellmer)
library(tibble)

paper <- "(...) A one-compartment model with linear elimination best
described methotrexate concentrations after i.v. infusion.
Typical clearance (CL) was 7.8 L/h (RSE 6%) and volume of
distribution (V) was 32.5 L (RSE 9%). (...)"

type_pk <- type_object (
  compound = type_string("Drug name" ),
  route    = type_string("Route of administration" ),
  V        = type_number("Volume of distribution in L" ),
  CL       = type_number("Clearance in L/h" )
)

chat <- chat_anthropic()
#> Using model = "claude-sonnet-5".

chat$chat_structured(paper, type = type_pk) |> as_tibble()
```

OUTPUT

```
# A tibble: 1 × 4
  compound      route      V      CL
  <chr>      <chr>    <dbl> <dbl>
1 methotrexate i.v. infusion 32.5    7.8
```

Key takeaways & Outlook

What we covered

- LLMs, harnesses, and agents: understanding the building blocks
- Adding PmX expertise through skills, subagents, and MCP tools
- Choosing between agentic flexibility and explicit orchestration

Outlook

- LLMs will fundamentally change how we work as pharmacometricians
- Realising their potential requires validation, human oversight, and evidence that earns our trust
- Shared benchmarks and continuous learning will help us build workflows we can trust

Thank you . . . →

Your questions

and anything you want to share